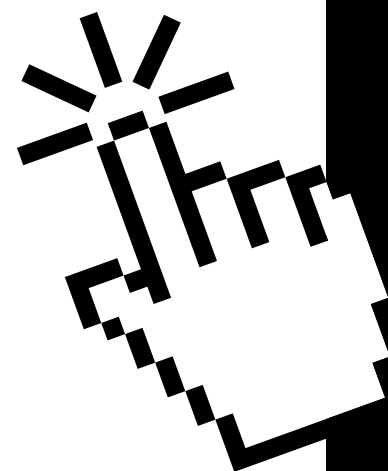




CP353002 Principles of Software Design and
Development



Trip Expense Splitter



นางสาว อภิตยา โคตรธีสาร



643021259-2 | Section 1



สารบัญ



✓ ที่มาและความสำคัญ

✓ วัตถุประสงค์

✓ ฟีเจอร์หลัก

✓ ขนาดของระบบ

✓ สถาปัตยกรรม

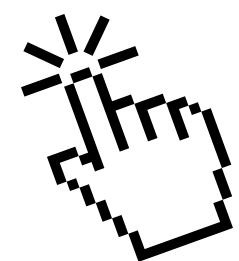
✓ Design Pattern

✓ หลักการ SOLID

✓ การทดสอบและ Deploy

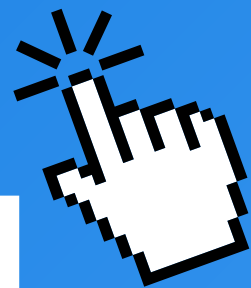
✓ ข้อจำกัดและต่อยอด

✓ ผู้จัดทำ





ที่มาและความสำคัญ



การเดินทางท่องเที่ยวเป็นกลุ่มต้องร่วมกันวางแผนเส้นทาง ที่พัก อาหาร และกิจกรรม แต่มักพบปัญหาข้อมูลกระจาย ตัดสินใจร่วมกันยาก และต้องคำนวณค่าใช้จ่ายด้วยมือ ระบบนี้จึงรวมทุกขั้นตอนไว้ในที่เดียว

ปัญหา

- ✓ ข้อมูลวางแผนทริปกระจายตามแชท
- ✓ ตัดสินใจร่วมกันยาก ไม่มีหลักฐานว่าเสียงส่วนใหญ่เลือกอะไร
- ✓ จดบิลและคำนวณว่าใครต้องโอนให้ใครด้วยมือ เสี่ยงผิดพลาด โดยเฉพาะบิลที่หารไม่เท่ากันหรือใช้หลายสกุลเงิน



แนวทางแก้

- ✓ ระบบเดียวครบ: วางแผน → หารบิล → สรุปยอด → โหวต → เช็กลิสต์ → ประวัติทริป
- ✓ เข้าร่วมด้วยรหัสเชิญ ใช้งานผ่านเว็บ



คุณค่าด้านวิศวกรรมซอฟต์แวร์

ประยุกต์ Layered Architecture, SOLID และ Design Pattern กับ ปัญหาจริง ครอบคลุมตั้งแต่ฐานข้อมูลจนถึง CI/CD และ Deploy



วัตถุประสงค์



ระบบครบวงจร

รวมการวางแผน การบิล สรุปยอด
โอน โหวต เช็กलिस्ट และประวัติทริป
ไว้ในระบบเดียว



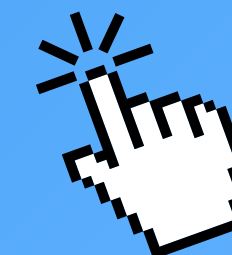
ออกแบบตามหลักการ

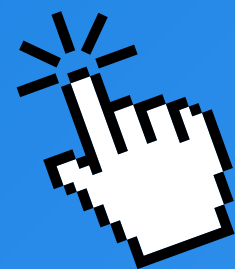
ประยุกต์ SOLID และ Design
Pattern กับปัญหาจริงในโค้ด



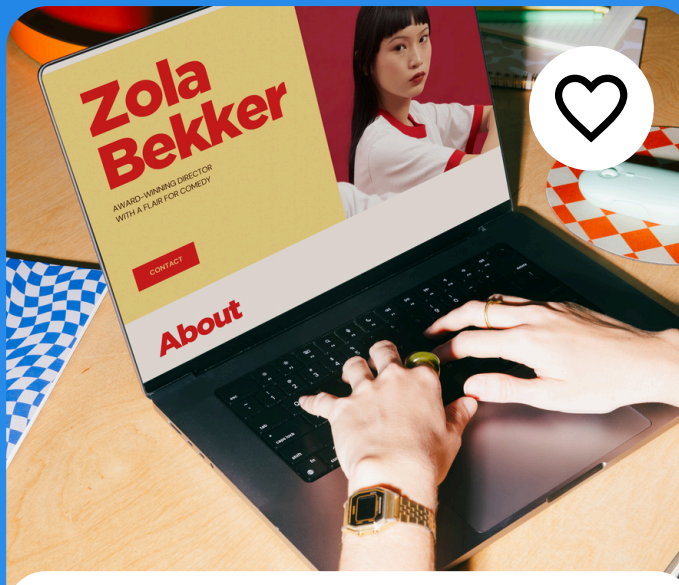
พร้อมใช้งานจริง

ออกแบบฐานข้อมูล ทดสอบ ทำ CI/CD
และ Deploy ให้ใช้งานผ่านเว็บได้



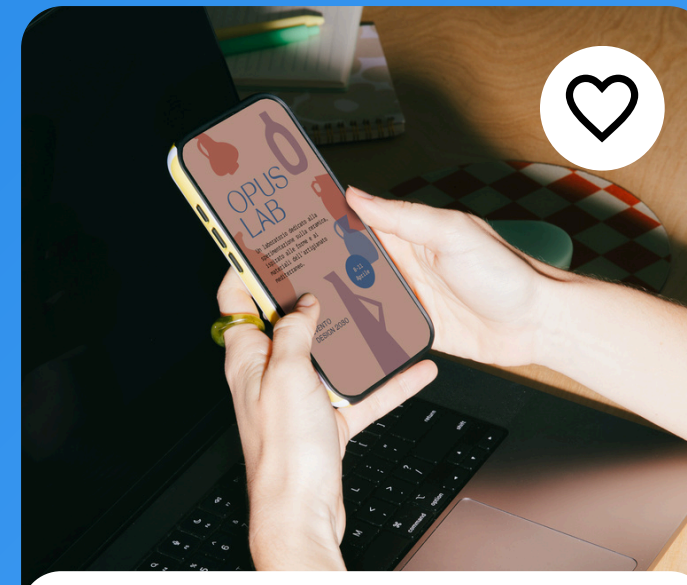


ฟีเจอร์หลัก



วางแผนทริป

- สร้างทริป
- ชวนเพื่อนด้วยรหัสเชิญ
- วางแผนกิจกรรมรายวัน : เดินทาง ที่พัก อาหาร และกิจกรรม



บิลและการหาร

- บันทึกบิล : หารเท่ากันหรือกำหนดยอดเอง สรุปว่าใครต้องโอนให้ใคร
- รองรับสกุลเงินต่างประเทศ



ตัดสินใจร่วมกัน

- โหวตเลือกร่วมกัน
- เช็กลิสต์พร้อมมอบหมายผู้รับผิดชอบ
- ดูประวัติความเคลื่อนไหวของทริป



ตาราง
ตารางในฐานข้อมูล

ฐานข้อมูลมี 18 ตาราง ครอบคลุมทริป สมาชิก บิล กิจกรรม โหวต และเช็กลิสต์ จัดการด้วย Flyway



FK
Foreign Key

มี Foreign Key 34 ตัว เลือก ON DELETE ตาม ความหมายของข้อมูล เช่น ห้ามลบข้อมูลที่เกี่ยวข้องกับ เงิน



GoF
Design Pattern

ใช้ 5 รูปแบบ ได้แก่ Singleton, Factory, Builder, Strategy และ Observer

ขนาดของระบบ

ระบบแบ่งเป็นชั้นชัดเจน ใช้ฐานข้อมูลเชิงสัมพันธ์ที่มีข้อกำหนดครบถ้วน และใช้ Design Pattern แก้ ปัญหาจริงในโค้ด



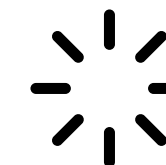


สถาปัตยกรรม



Controller

รับ request ตรวจสอบข้อมูล (@Valid) และสิทธิ์ แล้วเรียก Service ผ่าน interface



Service

ทำ business logic และ transaction เลือกวิธีการบิดผ่าน Strategy + Factory



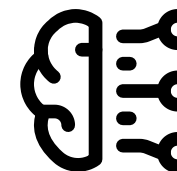
DTO + Mapper

แยก Entity ออกจาก API ไม่ให้พลัดล้มหลุด และตรวจสอบข้อมูลที่ขอบ API



Repository

Spring Data JPA เข้าถึง PostgreSQL ซ่อนรายละเอียดการเข้าถึงข้อมูล



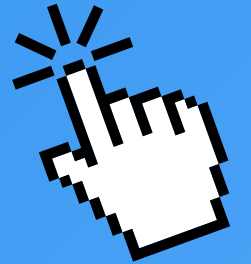
Config

AuthGuard ตรวจสอบสิทธิ์สมาชิก ApiErrorHandler ตอบ error เป็น JSON รูปแบบเดียว





Design Pattern



Singleton

ทุกคลาสที่มี @Service / @Component ผ่าน Spring Container (ไม่ได้เขียน getInstance เอง)



Factory

SplitStrategyFactory
เลือกวิธีการนับจากรหัส splitType โดยไม่มี if-else



Builder

ActivityResponse มี 28 ฟิลด์
ใช้ Builder กันใส่ค่าผิดตำแหน่ง



Strategy

EqualSplit / CustomSplit สลับวิธีการ
ได้ เพิ่มวิธีใหม่โดยไม่แก้ Service เดิม



Observer

Service ประกาศ event ให้
TripEventListener บันทึกประวัติทริป
โดยไม่รู้จักตารางประวัติ



Layered

Controller → Service → Repository
ชั้นบนเรียกชั้นล่างผ่าน interface



MVC

- Model = Entity + DTO
- View = HTML/JS เรียก REST
- Controller = @RestController

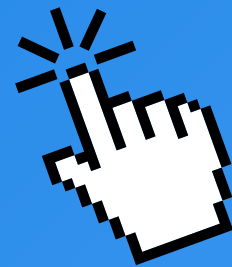


DI

Constructor Injection ทุกคลาส ไม่มี @Autowired ทดสอบ
ง่ายด้วย mock

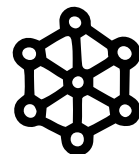


หลักการ SOLID



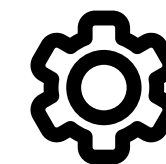
S และ O

- S : แยกหน้าที่ของแต่ละคลาส
- O : เพิ่มวิธีการใหม่ได้โดยไม่แก้ไขเดิม



L และ I

- L : Strategy ทุกตัวใช้แทนกันได้
- I : แยก interface ย่อยตามผู้ใช้ เช่น AuthTokenService มี 1 เมธอด



D

- Controller ขึ้นกับ interface ไม่ใช่คลาสจริง และใช้ Constructor Injection ทุกคลาส ไม่มี @Autowired



การทดสอบ

ทดสอบทั้งระดับหน่วยและยิง API จริง ผลรันจริงเมื่อ 7 ตุลาคม 2569 ผ่านทุกเคส ไม่มีเคสที่ไม่ผ่านหรือถูกข้าม

ผลการทดสอบ



👤 89

Unit Test ผ่านทั้งหมด



0

เคสที่ไม่ผ่าน

100%



API Test

Python ยิง API จริง
ครอบคลุมสิทธิ์ การใช้งาน
พร้อมกัน และฐานข้อมูล



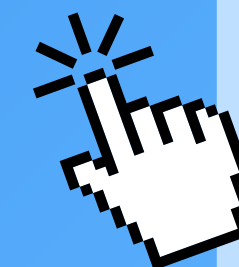
CI/CD

GitHub Actions รับ
build, unit test และ
docker build ทุก push
และ PR



Render + Neon

Deploy ใช้งานจริง





ข้อจำกัด และข้อยอด



ExpenseServiceImpl

ยาว 342 บรรทัด เมธอด apply
รวมหลายหน้าที่ ควรแยกเป็น
ExpenseValidator



Mapper และ Factory

ยังเป็นคลาสจริง ไม่มี
interface จึงไม่เข้มงวดตาม
DIP



Singleton / Factory

Singleton ใช้ผ่าน Spring
ส่วน Factory เป็น Simple
Factory ไม่ใช่ Factory
Method



Strategy ตัวที่ 3

PercentSplitStrategy (หาร
ตามเปอร์เซ็นต์) เป็นแนวทางต่อย
อด ยังไม่มีในโค้ด

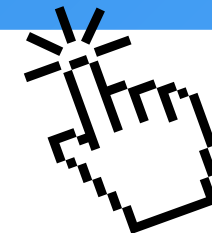


ข้อจำกัดการ Deploy

Render แพนฟรีกลับหลังไม่มีผู้
ใช้ ~15 นาที เปิดครั้งแรกรอนาน
(จำลอง CPU 0.1 core: เริ่มระบบ
~150 → ~90 วินาทีหลังตัด
dependency ที่ไม่ใช่)

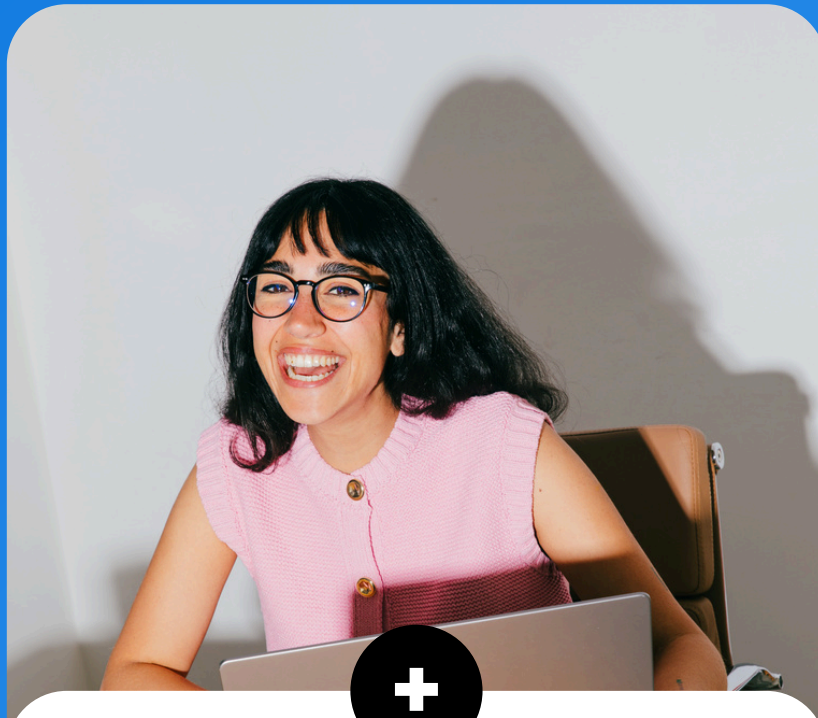


จุดที่ระบุไว้เอง
5 ข้อ



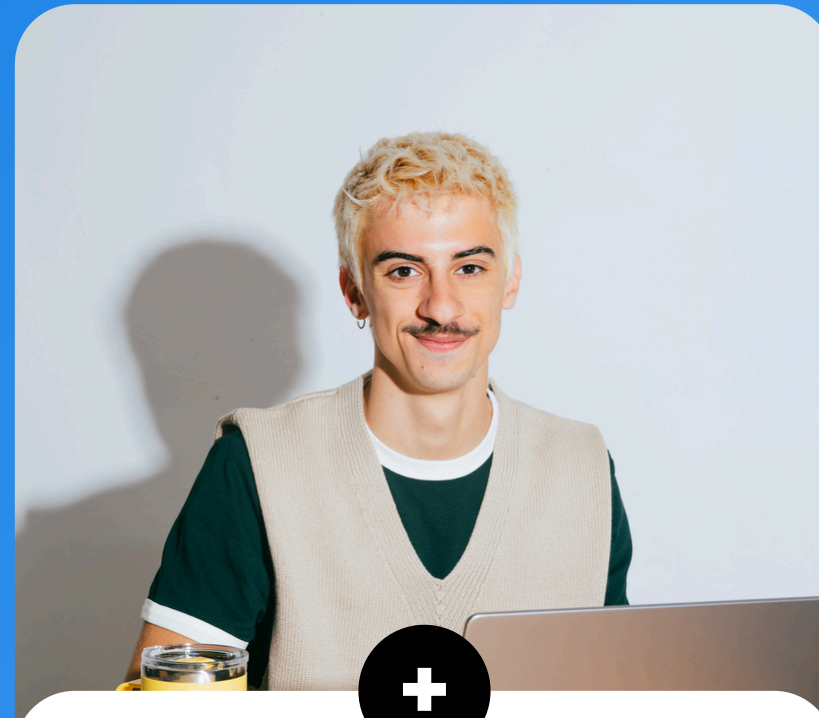


เครื่องมือ



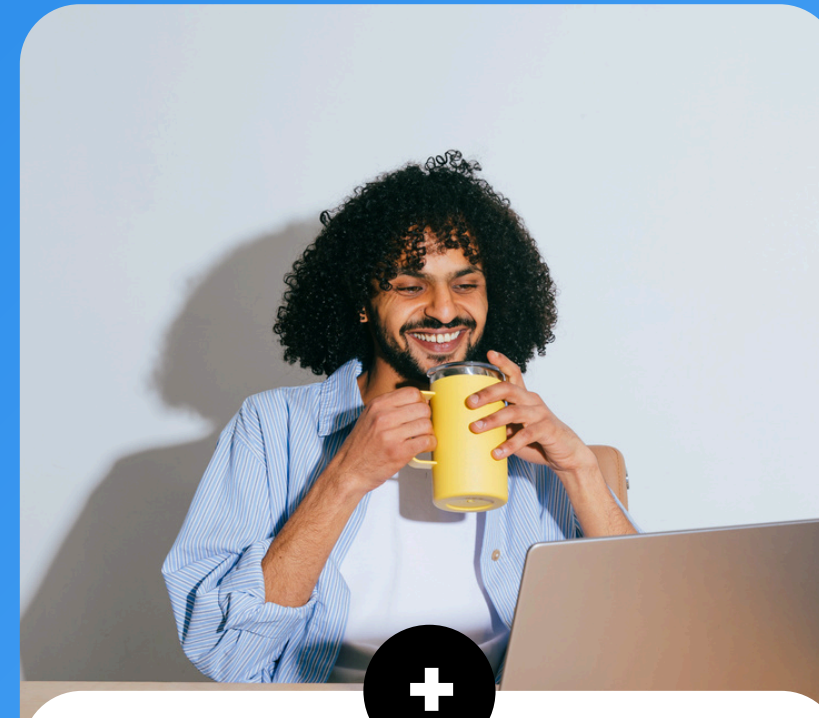
Backend

Spring Boot, REST
API



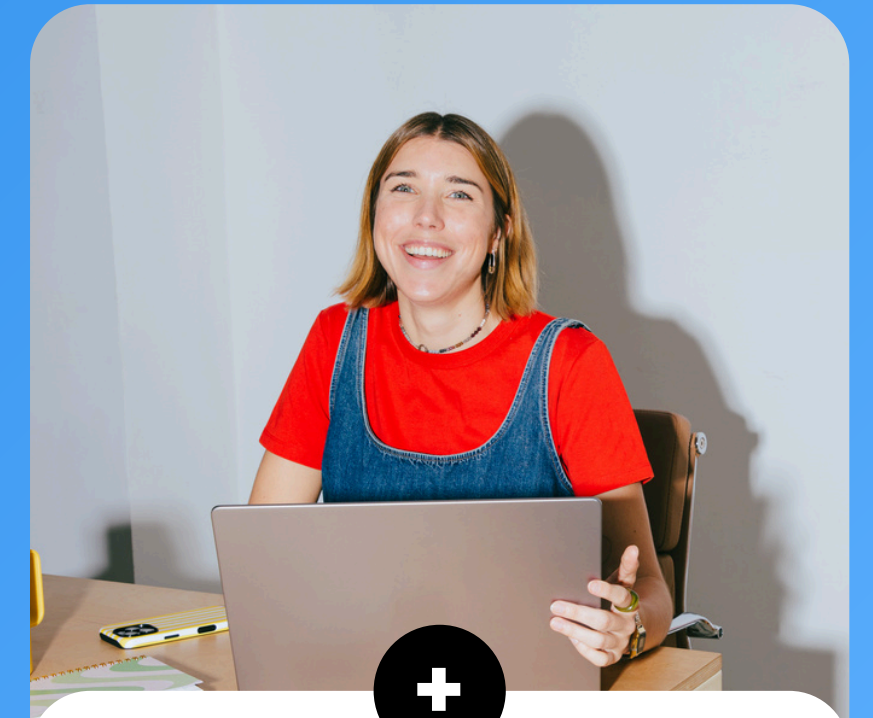
ฐานข้อมูล

ออกแบบ 18 ตาราง และ
Flyway



Frontend

HTML / CSS /
JavaScript



DevOps

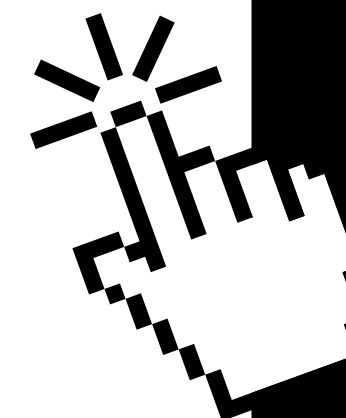
Test, Docker, CI/CD,
Deploy



Trip Expense Splitter



ขอบคุณ



643021259-2 | Section 1



trip-spitter.onrender.com



นางสาว อภิตยา โคตรธีสาร



github.com/txark/Trip_Spitter